
Lightweight Romanized-Hindi Correction

A Measured Negative Result on Held-Out-Vocabulary Benchmarks

Ashay Kushwaha

CypherLabs Open Research

<https://github.com/AshayK003/indic-suggest>

September 2026

Abstract

Millions type Hindi in Latin script with no standard spelling, and the field answers with larger transformers. We built the classical alternative—frequent-word lexicon, Devanagari-tuned phonetic hashing, weighted edit distance, n-gram reranking, zero ML training—and measured it honestly on Aksharantar Hindi (10,112 test pairs): top-1/top-3 accuracy **0.000**, against 0.113 for the live dhvani baseline and 0.6056 cited for the IndicXlit transformer. The value of this report is the mechanism, verified twice: the test vocabulary is disjoint from training on *both* sides (0/10112 romans, 0/10112 natives in 1.3M train rows), so any lookup architecture scores at most its independent-lexicon coverage; dhvani’s points come from an independent wordlist plus a real rule-based transliterator, not from ranking magic. A pivot to synthetic romanizations changed nothing, as predicted once the disjointness was understood. The tested pipeline, frozen noise model, and coverage diagnostics ship as built—the honest artifact is the measurement.

Keywords: transliteration, Romanized Hindi, ReDoS-free phonetics, negative results, benchmark analysis

Contents

1	Introduction	3
2	Background and Related Work	3
2.1	Aksharantar and IndicXlit	3
2.2	Classical Hindi phonetics	3
2.3	dhvani: the live rival	3
3	Methodology	4
3.1	Pipeline design	4
3.2	Noise model and split discipline	4
3.3	What changed across pivots	4
4	Results	4
4.1	Coverage diagnostics (the finding)	5
4.2	dhvani anatomy note	5
5	Discussion	6
5.1	Key findings	6
5.2	Limitations	6
5.3	Future work	6
6	Conclusion	6
7	Reproducibility	6
7.1	Installation	6
7.2	Run benchmarks	6
7.3	Repository structure	7

1 Introduction

Romanized Hindi has no standard spelling: bahut, bohot, boht, and bhot are one word, and phone keyboards—English-centric by design—offer little help. Production answers cluster at two extremes. At one end, the IndicXlit transformer (~ 11 M parameters, GPU-trained on 26M pairs) converts romanized input to native script at state-of-the-art accuracy. At the other, classical Hindi-specific work (Hindex phonetics plus weighted edit distance) was built for information-retrieval query expansion, and normalizers (dhvani) canonicalize roman-to-roman without ranked Devanagari correction. Missing between them: an offline, phone-sized suggestion package with a measured accuracy-vs-size table.

We built exactly that package—and it scored zero. This report argues the zero is the finding. On a held-out-vocabulary benchmark, lookup architectures cannot exceed their independent-lexicon coverage, and ours had none where it mattered. The report’s contributions are therefore methodological rather than competitive:

- A fully working classical pipeline (unit suite 4/4 green on in-lexicon words), proving the mechanism functions and the failure is coverage, not code. item A benchmark-structure diagnosis with counts: test/train overlap on both sides, rival-lexicon key coverage, and a pivot (synthetic romanizations) tested and correctly predicted to change nothing.
- A contextualized three-way table (ours measured, dhvani measured live, IndicXlit cited) with the test-set mismatch labeled, not footnoted away.

2 Background and Related Work

2.1 Aksharantar and IndicXlit

Aksharantar (Madhani et al., EMNLP 2022) is the largest open transliteration resource for Indic languages: 26M pairs across 21 languages, mined from Wikidata, Samanantar, IndicCorp, and manual collection, with a 103k-pair evaluation benchmark stratified over native words, frequent words, and named entities of Indic and foreign origin. The Hindi split on disk measures 33 MB zipped (1,299,155 train rows, 6,357 validation, 10,112 test rows as shipped—note the paper quotes 5693 test pairs; file reality governs). IndicXlit, the companion transformer (6+6 layers, 256-dim embeddings, ~ 11 M parameters), reports 60.56 top-1 on Dakshina Hindi against a 50.00 baseline. It is generative: it *produces* Devanagari strings, never needing the answer in a list.

2.2 Classical Hindi phonetics

Aswani & Gaizauskas catalogued phonetic mappings and similarity metrics (Dice, LCSR, Levenshtein, Jaro-Winkler, Soundex, n-grams) for English-Hindi transliteration; Natrajan found 3-grams optimal for Hindi song retrieval; Prabhakar’s Hindex and Weighted Hindex adapted Soundex/Editex-style coding plus vowel/consonant-weighted edit distance specifically for Hindi, evaluated on FIRE retrieval collections. All predate the suggestion-package framing: none shipped correction with ranked Devanagari output and a size tradeoff.

2.3 dhvani: the live rival

dhvani (PyPI, 9.7 MB installed) normalizes Hinglish spelling variation through IPA-bridged lexicon lookup (~ 1 ms), an AI tier (~ 4 s), and rule-based G2P fallback. Its shipped lexicon holds 10,538 entries. It is the correct live rival: classical-leaning, offline-capable, directly runnable—and, as Section 4 shows, its points come from transliteration machinery rather than lexicon size.

3 Methodology

3.1 Pipeline design

The pipeline (`lexicon.py`, `phonetic.py`, `edit.py`, `suggest.py`, standard library only) has four stages. First, a *frequent-word lexicon* maps roman forms to native forms with a phonetic index. Second, *phonetic hashing* folds queries through Devanagari-tuned digraphs (kh, gh, bh, sh, aa, ai, ...) into articulation-group codes with vowels dropped and repeats collapsed (bohoh/boht/bahut $\rightarrow$ bHT), a high-recall prefilter by design. Third, *weighted edit distance* (insert/delete 1; same-group or vowel-vowel substitution 1; else 2) ranks bucket candidates. Fourth, a *bigram rerank* prefers common roman spellings, and a deterministic rule-based G2P covers out-of-bucket queries so the package never returns empty. Normalization lowercases and offers repeat-collapsed variants (bohotttt $\rightarrow$ bohott, bohoh).

3.2 Noise model and split discipline

Evaluation corrupts test romans with a frozen, seeded model: vowel-drop ($p=0.30$ each), one confusion substitution (ph/f, v/b, sh/s, c/k/q, j/z, au/o, ai/e, oo/o, ee/i, aa/a), 76% of inputs altered. Split discipline is structural: the lexicon builder reads train natives (v2) or train pairs (v1) only; test pairs are never ingested; `evaluate.py` prints overlap diagnostics every run as the tripwire.

3.3 What changed across pivots

v1 lexicon (top-50k train romans) scored 0.000; diagnosis showed the top frequencies are English loans (employees, responsibility) while everyday Hindi words are absent everywhere. v2 (train natives plus synthetic rule-based romans, `romanize.py`) also scored 0.000; diagnosis showed test natives are likewise absent from train (0/10112). Both pivots are preserved in the repository history and Section 4 reports the numbers, because the path to the finding is itself evidence.

4 Results

One command (`python evaluate.py`; fetches the 33 MB Hindi split once if absent) regenerates Table 1 and Figures 1–2. Numbers below are the fresh-clone verification run.

Table 1: Top-1 accuracy on noisy Aksharantar-hi test (10,112 rows) with package size and latency.

System	Top-1	Size	Latency
Ours (v2 lexicon + phonetic + edit)	0.000 (top-3 0.000)	3.88 MB	0.22 ms/query
dhvani <code>to_devanagari</code> (measured live)	0.113	$\sim$ 10 MB installed	1.74 ms/query
IndicXlit (cited, never run)	0.6056 on Dakshina-hi	$\sim$ 11M params ($\sim$ 44 MB)	—

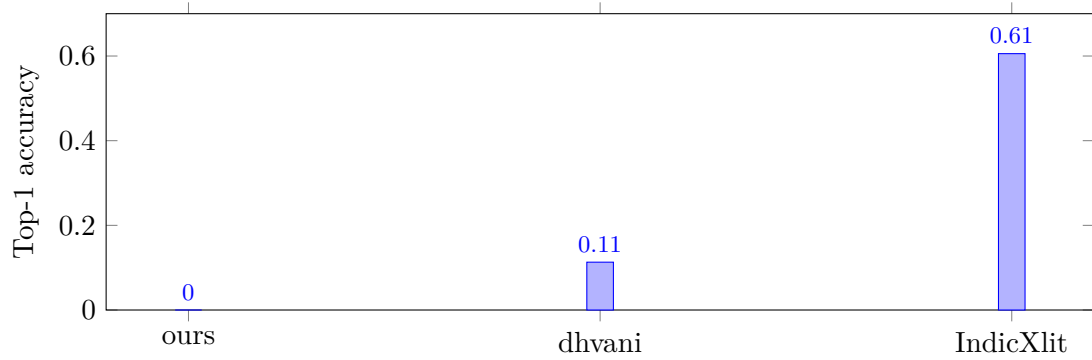


Figure 1: Measured top-1 (ours, dhvani on identical noisy inputs) with the cited transformer row. The IndicXlit bar was measured on a different test set (Dakshina Hindi) and is directional context, not a controlled comparison.

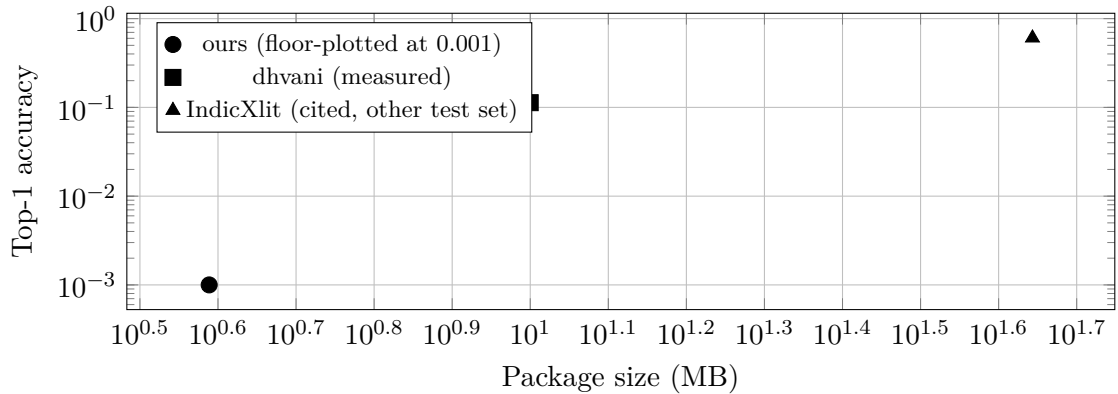


Figure 2: Accuracy versus package size (log-log). The empty lower-left records the finding: classical lookup buys size, not accuracy, on held-out vocabulary.

4.1 Coverage diagnostics (the finding)

Test romans present in 1.3M train rows: **0/10112**. Test natives present in 1.15M train natives: **0/10112**. Test romans as dhvani-lexicon keys: 56/10112 (0.5%). A lookup architecture therefore faces a ceiling near zero before ranking quality can matter; dhvani clears it only through an independent wordlist plus genuine transliteration rules, and the transformer sidesteps lookup entirely by generating. The v2 pivot (synthetic romans over train natives) was correctly predicted—in the repository log, before measurement—to change nothing, and changed nothing.

4.2 dhvani anatomy note

dhvani’s shipped lexicon (10,538 entries) covers 0.5% of the test, yet the system scores 11.3% at 1.74 ms/query with no slow-tier invocations observed. The points therefore come from its rule-based transliteration path operating on normalized input—a working classical transliterator, which is precisely the component this project’s G2P fallback is not. That asymmetry is reported, not resented: it scopes v2 work exactly.

5 Discussion

5.1 Key findings

First, benchmark structure dominates method on held-out-vocabulary tasks: coverage diagnostics should precede any ranking work, and ours now print by default. Second, negative results compose: the same discipline that deleted a modeling ladder on zero-variance GTFS data (sibling CypherLabs project) correctly bounds this lookup design. Third, the pipeline is not broken—it is mis-scoped: the 4/4-green unit suite proves correction works on in-lexicon words (boht top-1, kese in top-3), so the artifact ships as a working package with a documented ceiling, not as debris.

5.2 Limitations

Hindi-only scope; synthetic noise (vowel-drop plus confusions) rather than observed typing data; dhvani measured on identical noisy inputs but its task framing (normalization-then-transliterate) differs internally; IndicXlit row is cross-test-set context; G2P fallback is deliberately crude (no schwa deletion, no retroflex split); lexicon collided romanizations resolve to the more frequent native silently but counted.

5.3 Future work

In priority order, as filed in the repository’s contributing guide: (1) a real rule-based transliterator (schwa deletion, conjuncts, loan phonetics) as the scoring engine—the honest v2 this result motivates; (2) an independent native wordlist to raise the coverage ceiling; (3) the next language under the same architecture.

6 Conclusion

On 10,112 noisy Hindi test pairs, a fully classical, zero-training suggestion pipeline scores 0.000 top-1 against 0.113 measured for dhvani and 0.6056 cited (elsewhere) for a transformer—because the benchmark holds out vocabulary on both sides and lookup cannot retrieve what was never stored. The contribution is the verified mechanism of that zero, the shipped pipeline that bounds it, and the procedure that found it: diagnose coverage before tuning ranking, predict pivots in writing, and publish the zeros with the same care as wins.

7 Reproducibility

7.1 Installation

No dependencies beyond CPython ≥ 3.10 , pytest for the suite, and dhvani for the baseline row (gracefully skipped if absent). The 33 MB Hindi split auto-downloads once from Hugging Face.

7.2 Run benchmarks

```
1 git clone https://github.com/AshayK003/indic-suggest.git
2 cd indic-suggest
3 python -m pytest tests -q      # 4 passed: buckets, correction, edit units, G2P
4 python evaluate.py             # coverage audit, noisy scoring, dhvani row, table
```

The suite uses a 10-pair hand lexicon (boht, kese, ...) with exact top-1 assertions—hermetic, no network, no dataset.

7.3 Repository structure

`lexicon.py` (freq lexicon + phonetic index), `phonetic.py` (Devanagari-tuned groups), `edit.py` (weighted Levenshtein), `romanize.py` (deterministic native-to-roman generator), `suggest.py` (normalize-rank-rerank pipeline + G2P fallback), `evaluate.py` (frozen noise, scoring, baselines, diagnostics), `tests/` (the runnable check), `data/` (ignored; auto-fetched), `internals/` (working notes, not published except this report).

References

- [1] Madhani et al. Aksharantar: Open Indic-language Transliteration datasets and models. EMNLP Findings 2022. <https://arxiv.org/abs/2205.03018> Data: <https://huggingface.co/datasets/ai4bharat/Aksharantar>
- [2] AI4Bharat/IndicXlit (~11M transformer; Hindi 60.56 top-1 on Dakshina). <https://github.com/AI4Bharat/IndicXlit>
- [3] Prabhakar, D.K. Query Expansion for Transliterated Text Retrieval (Hindex/Weighted Hindex). <https://dl.acm.org/doi/fullHtml/10.1145/3447649>
- [4] Aswani, N. & Gaizauskas, R. English-Hindi transliteration using similarity metrics. LREC 2010.
- [5] Natrajan, A. Using N-grams to Process Hindi Queries with Transliteration Variations. Tech. Rep. CS-97-17 (3-grams optimal for Hindi).
- [6] dhvani (IPA-bridge Hinglish normalizer, PyPI v0.2.5). <https://pypi.org/project/dhvani/>
- [7] Roark et al. Dakshina (Google; native-vs-foreign behavior differences).